CSWSH | Cross-Site WebSocket Hijacking | מעבדה מקומית בלבד

דוח מעבדה – Cross-Site WebSocket Hijacking (CSWSH)

דוח זה מציג את הרקע התאורטי לחולשת CSWSH, את מימוש ה־POC בסביבת מעבדה מקומית, את שלבי הניצול בפועל, את תוצאות ההדגמה, וכן את מנגנוני ההגנה שנוספו בגרסה המאובטחת.

שמות המגישים

Yousef Salman, Salah Shamasaneh

תאריך

27/02/2026

נושא

Track A – CSWSH

תוכן עניינים

1. תיאור החולשה והרקע התאורטי
2. סביבת העבודה
3. מימוש והדגמת המתקפה (POC)
4. מנגנוני הגנה ומניעה
5. מקורות

תיאור החולשה והרקע התאורטי

מהי החולשה

Cross-Site WebSocket Hijacking היא חולשה שבה אתר חיצוני מצליח לגרום לדפדפן של משתמש שכבר מחובר לאפליקציה לגיטימית לפתוח חיבור `WebSocket` אל שרת היעד. אם השרת מקבל את החיבור על בסיס session cookie בלבד, וללא בדיקת `Origin` או אימות ייעודי לחיבור, הוא עלול להתייחס לחיבור כאילו נוצר מתוך האפליקציה האמיתית.

כיצד היא נוצרת

- המשתמש כבר מחובר לאפליקציה הלגיטימית ולכן יש בדפדפן session cookie תקף.
- אתר חיצוני מפעיל JavaScript שפותח `WebSocket` אל שרת היעד.
- השרת לא בודק את מקור הבקשה (Origin) ולא דורש token נפרד לחיבור.
- השרת סומך על ה-session בלבד ולכן מאשר את החיבור ומשייך אותו למשתמש המחובר.

באיזה הקשר היא מופיעה במערכות ווב

החולשה רלוונטית במיוחד למערכות עם תקשורת בזמן אמת כמו צ'אט, התראות, dashboards, ניטור בזמן אמת, מערכות תמיכה ומשחקים. בכל מצב שבו המשתמש מחובר והאפליקציה משתמשת ב-WebSocket, חוסר בבדיקות צד שרת עלול לאפשר ניצול.

מה תוקף יכול להשיג באמצעות החולשה

- שליחת הודעות בשם המשתמש המחובר.
- ביצוע פעולות שינוי מצב אם הן זמינות דרך הערוץ.
- במקרים מסוימים – קבלת מידע בזמן אמת שיועד למשתמש.

למה זה שונה מ־HTTP רגיל

ב־HTTP מדובר לרוב בבקשות נפרדות, בעוד שב־WebSocket יש handshake אחד שמוביל לערוץ דו־כיווני מתמשך. אם ההרשאה נעשתה בצורה חלשה בתחילת החיבור, הבעיה נמשכת לכל אורך הסשן של ה־WebSocket.

סביבת העבודה

טכנולוגיות

רכיב	בחירה
שפת תכנות	Node.js / JavaScript

Backend	Express, ws, express-session
Client	HTML, CSS, JavaScript
סביבת עבודה	VS Code, Git Bash, דפדפן Chrome/Edge
פריסת המעבדה	Victim App על פורט `3000`, Attacker Site על פורט `4000`, Secure Victim App על פורט `5000`, Launcher על פורט `8080`

כלים המשמשים לביצוע ההדגמה

- Browser Developer Tools לבדיקת Console, Network ו־WebSocket activity.
- Terminal להצגת לוגים בזמן אמת מכל אחד מהשרתים.
- שרת השקה מרכזי (Launcher) לפתיחה מהירה של שלוש הסביבות.

מימוש והדגמת המתקפה (POC)

תיאור של ה־POC

בפרויקט נבנתה אפליקציית Victim הכוללת מסך Login וצ'אט בזמן אמת. לאחר התחברות, המשתמש מקבל session cookie והלקוח יוצר חיבור WebSocket לצורך שליחת הודעות. בגרסה הפגיעה, שרת ה־Victim מאפשר את החיבור על בסיס ה־session בלבד ואינו בודק אם החיבור הגיע מ־Origin לגיטימי. בנוסף נבנה Attacker Site בפורט נפרד שמטרתו לגרום לדפדפן של המשתמש לפתוח חיבור WebSocket אל ה־Victim.

שלבי הניצול

1. המשתמש נכנס ל־Victim App דרך `http://localhost:3000/login`.
2. השרת יוצר session ושומר cookie בדפדפן.
3. המשתמש פותח את מסך הצ'אט והחיבור הלגיטימי ל־WebSocket עובד כרגיל.
4. באותו דפדפן, המשתמש פותח את Attacker Site ב־`http://localhost:4000`.
5. ה־Attacker page מפעיל `new WebSocket('ws://localhost:3000/ws')`.
6. הדפדפן שולח את ה־session cookie הקיים כחלק מ־handshake.
7. בגלל שהשרת הפגיע לא בודק Origin ולא דורש token ייעודי, הוא מאשר את החיבור.
8. האתר התוקף שולח payload מסוג `chat`.
9. השרת מעבד את הפעולה כאילו בוצעה מתוך האפליקציה הלגיטימית, בשם המשתמש המחובר.

Payloads / קוד / בקשות לדוגמה

```
{
  "type": "chat",
  "text": "This message was sent from the attacker page."
}
```

```
const ws = new WebSocket('ws://localhost:3000/ws');
ws.onopen = () => {
  ws.send(JSON.stringify({
    type: 'chat',
    text: 'This message was sent from the attacker page.'
  }));
};
```

תוצאה בפועל של הניצול

תוצאה צפויה בגרסה הפגיעה

- בלוגים של השרת מופיע `Origin: http://localhost:4000`.
- למרות זאת, השרת עדיין מזהה session תקף ומאשר את החיבור.
- הודעת ה־chat נשלחת ומוצגת כאילו הגיעה מהמשתמש האמיתי.

דיאגרמה המתארת את שלבי המתקפה

1 **התחברות לגיטימית ל־Victim App:** המשתמש נכנס דרך `http://localhost:3000/login` והשרת יוצר session תקף.

↓

2 **שמירת ה־session בדפדפן:** הדפדפן שומר את ה־cookie ומשתמש בו בהמשך מול השרת הלגיטימי.

↓

3 **טעינת אתר התוקף:** המשתמש פותח את `http://localhost:4000` באותו דפדפן.

↓

4 **פתיחת WebSocket מתוך הדפדפן:** JavaScript בדף התוקף מפעיל `new WebSocket('ws://localhost:3000/ws')`.

↓

5 **השרת הפגיע מקבל את החיבור:** מאחר שאין בדיקת `Origin` ואין token ייעודי, השרת מאשר את החיבור על בסיס ה־session cookie.

↓

6 **שליחת payload ועיבודו בשם המשתמש:** אתר התוקף שולח הודעת chat, והשרת מעבד אותה כאילו נשלחה מתוך האפליקציה הלגיטימית.

הדיאגרמה מתארת זרימת רצף מלאה: התחברות לגיטימית, טעינת דף תוקף, פתיחת WS מתוך הדפדפן, ואישור חיבור בשרת הפגיע בשל היעדר בדיקות מקור ואימות ייעודי.

מנגנוני הגנה ומניעה

מדוע ההגנות הקיימות נכשלו

- השרת סמך רק על ה־session cookie כדי לזהות את המשתמש.
- לא בוצעה בדיקת `Origin` במהלך ה־WebSocket handshake.
- לא היה token נפרד לחיבור WebSocket.
- לא הייתה מדיניות deny-by-default להודעות/פעולות.

כיצד ניתן למנוע את החולשה

1. בדיקת Origin בצד השרת

יש לאשר חיבורי WebSocket רק מתוך רשימת origins מורשים. בגרסה המאובטחת בפרויקט, רק `http://localhost:5000` מאושר להתחבר לשרת המאובטח.

2. אימות חיבור בעזרת token ייעודי

לאחר פתיחת החיבור, הלקוח שולח הודעת `auth` עם token שמקושר ל-session. אם ה־token שגוי או חסר – החיבור נסגר.

3. הרשאות ברמת הודעה

השרת צריך לאשר רק סוגי הודעות ידועים ולדחות כל פעולה שאינה מוגדרת. כך גם אם נוצר חיבור, לא ניתן לבצע כל פעולה שרירותית.

4. ניהול session וחיבורים

מומלץ לנתק חיבורי WebSocket כאשר המשתמש מתנתק, כאשר ה־session פג, ולמנוע שימוש בנתונים רגישים ב־logs או בפרמטרים של URL.

Best Practices רלוונטיים

- Origin allowlist קשיח לכל endpoint של WebSocket.
- Token או challenge-response נפרד לחיבור WS ולא רק cookie.
- אימות ולידציה לכל payload, כולל הגבלת אורך הודעות.
- deny-by-default לכל message type שלא הוגדר במפורש.
- סגירת חיבורים פעילים בעת logout או session expiration.

מה רואים בגרסה המאובטחת

- כאשר ה־Attacker Site מנסה להתחבר ל־`ws://localhost:5000/ws`, השרת מחזיר דחייה.
- בלוגים מציגים שניסיון החיבור הגיע מ־Origin לא מורשה.
- ללא token תקף, גם חיבור שנפתח לא יוכל לבצע פעולות.

מקורות

- OWASP WebSocket Security Cheat Sheet – https://cheatsheetseries.owasp.org/cheatsheets/WebSocket_Security_Cheat_Sheet.html
- MDN – Writing WebSocket servers – https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API/Writing_WebSocket_servers
- RFC 6455 – The WebSocket Protocol – https://www.rfc-editor.org/rfc/rfc6455
- OWASP CSRF Prevention Cheat Sheet – https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

המסמך מיועד לצרכי לימוד והדגמה בסביבת מעבדה מקומית בלבד.